

Problem Set 4

Fundamentals of Numerical Programming I

LEONARD STORCKS

January 10, 2026

Problem 1. Convergence of Newton's root finding method

In the lecture we have stated:

If g is continuously differentiable in a neighborhood around a root x^* with nonzero derivative at x^* , then there exists a neighborhood around x^* where Newton's iteration converges to x^* for all starting values.

Now consider the function

$$g(x) = x(1 + x^2)^{-\frac{1}{2}} \tag{1}$$

- (a) What is the root x^* of g ?
- (b) Calculate $g'(x^*)$. What does this imply for Newton iteration?
- (c) Explicitly calculate the newton update step

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} \tag{2}$$

- (d) Consider the following cases for the initialization of the Newton iteration

- $x_0 = 0.5$
- $x_0 = -1.0$
- $x_0 = 2.0$

and analyze if the iteration converges, diverges or oscillates.

Problem 2. Gradient descent vs Newton optimization

In this task we are going to consider three different functions

- the quadratic form

$$L_1(\underline{\theta}) = \frac{1}{2} \underline{\theta}^T \underline{A} \underline{\theta} + \underline{\theta}^T \underline{b} \quad (3)$$

where $\underline{A} \in \mathbb{R}^{p \times p}$ is positive definite

- *Himmelblau's function*

$$L_2(\underline{\theta}) = (\theta_1^2 + \theta_2 - 11)^2 + (\theta_1 + \theta_2^2 - 7)^2, \quad \underline{\theta} = \begin{pmatrix} \theta_1 \\ \theta_2 \end{pmatrix} \quad (4)$$

- and the *Goldstein-Price function*

$$L_3(\underline{\theta}) = [1 + (\theta_1 + \theta_2 + 1)^2(19 - 14\theta_1 + 3\theta_1^2 - 14\theta_2 + 6\theta_1\theta_2 + 3\theta_2^2)] \\ [30 + (2\theta_1 - 3\theta_2)^2(18 - 32\theta_1 + 12\theta_1^2 + 48\theta_2 - 36\theta_1\theta_2 + 27\theta_2^2)] \quad (5)$$

(a) Implement gradient descent

$$\begin{aligned} \underline{\theta}^{(n+1)} &= \underline{\theta}^{(n)} - \gamma \underline{\nabla} L|_{\underline{\theta}=\underline{\theta}^{(n)}}, \quad \text{learning rate } \gamma \\ \text{until } &\left| L(\underline{\theta}^{(n+1)}) - L(\underline{\theta}^{(n)}) \right| < \epsilon \text{ or } \# \text{ steps} > N_{\max} \end{aligned} \quad (6)$$

in Python. Your gradient descent function should take as input

- the starting point $\underline{\theta}^{(0)}$
- the objective function $L(\underline{\theta})$
- its gradient function $\underline{\nabla} L(\underline{\theta})$
- the learning rate γ
- the tolerance ϵ
- and the maximum number of iterations $N_{\max}$

and should return all iteration steps.

(b) For

$$\underline{A} = \begin{pmatrix} 10 & 0 \\ 0 & 1 \end{pmatrix}, \quad \underline{b} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad (7)$$

plot the contours and heatmap of L_1 for $\theta_1 \in [-5.0, 5.0], \theta_2 \in [-2, 2]$. For

$$\underline{\theta}^{(0)} = \begin{pmatrix} -4 \\ 1.5 \end{pmatrix}, \quad \gamma \in \{0.05, 0.01, 0.018\}, \quad \epsilon = 10^{-4} \quad (8)$$

plot the iteration steps. What do you observe?

(c) Implement Newton optimization in Python. Your Newton optimization function should take as input

- the starting point $\underline{\theta}^{(0)}$
- the objective function $L(\underline{\theta})$
- its gradient function $\underline{\nabla} L(\underline{\theta}) \in \mathbb{R}^p$
- its Hessian function $\underline{\underline{H}}_L(\underline{\theta}) \in \mathbb{R}^{(p \times p)}$
- the tolerance ϵ
- and the maximum number of iterations $N_{\max}$

and should return all iteration steps.

(d) Apply Newton optimization to L_1 with

$$\underline{\theta}^{(0)} = \begin{pmatrix} -4 \\ 1.5 \end{pmatrix}, \quad \epsilon = 10^{-4} \quad (9)$$

How many iterations are necessary?

- (e) What is the unique minimizer of L_1 with positive definite $\underline{\underline{A}}$?
- (f) Show that this minimizer is reached in a single Newton optimization step. Why doesn't it make sense to apply Newton optimization to L_1 ?
- (g) Now consider L_2 . Plot the contours and heatmap of L_2 for $\theta_1, \theta_2 \in [-5, 5]$.
- (h) Calculate the gradient and Hessian of L_2 and implement them in Python.
- (i) Apply both gradient descent and Newton iteration on L_2 for

$$\underline{\theta}^{(0)} \in \left\{ \begin{pmatrix} 0.0 \\ -4.0 \end{pmatrix}, \begin{pmatrix} -0.25 \\ -0.9 \end{pmatrix} \right\}, \quad \epsilon = 10^{-4} \quad (10)$$

- (j) (*Optional*) Plot the contours and heatmap of L_2 for $\theta_1 \in [-2, 2], \theta_2 \in [-3, 1]$. Our aim is to find the minimum of L_2 using gradient descent. However, calculating the gradient

will be tedious. Try using finite differencing for estimating the gradient. Alternatively you might also look into automatic differentiation using **JAX**.

Problem 3. (*Optional*) Gauss-Newton for nonlinear curve fitting

Consider we are given data

$$\begin{aligned}\underline{t} &= \begin{pmatrix} 0.0 & 0.1 & 0.2 & 0.3 & 0.4 & 0.5 & 0.6 & 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 & 1.3 & 1.4 & 1.5 & 1.6 & 1.7 & 1.8 & 1.9 \end{pmatrix}^T \in \mathbb{R}^{20} \\ \underline{y} &= \begin{pmatrix} 2.55 & 2.21 & 2.05 & 1.93 & 1.57 & 1.42 & 1.47 & 1.28 & 1.06 & 1.07 \\ 0.90 & 0.84 & 0.85 & 0.59 & 0.57 & 0.65 & 0.58 & 0.69 & 0.54 & 0.47 \end{pmatrix}^T \in \mathbb{R}^{20}\end{aligned}\quad (11)$$

and want to fit parameters in the model

$$f_{\underline{\theta}}(t) = \theta_0 + \theta_1 \exp(-\theta_2 t) \quad (12)$$

to minimize

$$\underline{\theta}^* = \underset{\underline{\theta}}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^N \underbrace{(f_{\underline{\theta}}(t_i) - y_i)^2}_{r_i} = \frac{1}{2} \|\underline{r}(\underline{\theta})\|_2^2, \quad \underline{r}(\underline{\theta}) = (r_1(\underline{\theta}), \dots, r_N(\underline{\theta}))^T \quad (13)$$

(a) Implement the residual function $\underline{r}$ in Python. Its inputs are

- the model $f_{\underline{\theta}}$
- the parameters $\underline{\theta} \in \mathbb{R}^p$
- the evaluation points $\underline{t} \in \mathbb{R}^N$
- the data $\underline{y} \in \mathbb{R}^N$

(b) Calculate and implement the Jacobian

$$\underline{J}_{\underline{r}}(\underline{\theta}) = \left[\frac{\partial r_j}{\partial \theta_i} \right]_{\substack{j=1, \dots, N \\ i=1, \dots, p}} \in \mathbb{R}^{N \times p} \quad (14)$$

for our specific model $f_{\underline{\theta}}(t)$ and an arbitrary input vector $\underline{t} = (t_1, \dots, t_N)^T$.

(c) Implement Gauss-Newton iteration

$$\underline{\theta}_{k+1} = \underline{\theta}_k - \left(\underline{J}_{\underline{r}}(\underline{\theta}_k)^T \underline{J}_{\underline{r}}(\underline{\theta}_k) \right)^{-1} \underline{J}_{\underline{r}}(\underline{\theta}_k)^T \underline{r}(\underline{\theta}_k) \quad (15)$$

and apply it to our test data. Plot the results. *Hint:* Do not explicitly invert

$J(\theta_k)^T J(\theta_k)$. Instead, we rewrite

$$\theta_{k+1} = \theta_k + p_k \quad (16)$$

with

$$p_k = - \left(J(\theta_k)^T J(\theta_k) \right)^{-1} J(\theta_k)^T r(\theta_k) \quad (17)$$

which we can write as the linear problem

$$\left(J(\theta_k)^T J(\theta_k) \right) p_k = -J(\theta_k)^T r(\theta_k) \quad (18)$$

You can solve this linear problem using `scipy.linalg.solve`.